

Rapport de Projet Multidisciplinaire

Modélisation du Quadrupôle d'Okayama par BEM

(Boundary Element Method)

Zoé NONON — Louise LAMMERTYN

4^{ème} année Génie Physique

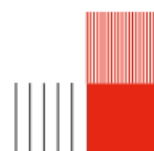
Tuteurs :

Florent HOUELLIER
Lucile BERDOUX DURIEUX
Jean-Baptiste MELLIER
Pier-Francesco FAZZINI

Année universitaire 2025-2026

Institut National des Sciences Appliquées de Toulouse

5 mars 2026



Résumé

Ce document est le rapport final du projet multidisciplinaire de 4ème année en Génie Physique de l'INSA Toulouse. Il a pour but d'expliquer en détail la démarche que nous avons suivie afin de mener à bien ce projet. Ce document retrace donc le contexte, l'organisation mais aussi toutes les équations que nous avons dû étudier.

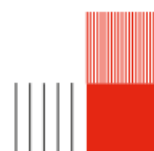
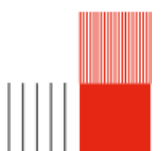


Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Présentation du projet	2
1.3	Objectifs du projet	3
1.3.1	Objectif 1 - Prise en main de BEMPP	3
1.3.2	Objectif 2 - Algorithme de la décomposition multipolaire	3
1.3.3	Objectif 3 - EXtraction du potentiel	3
1.3.4	Objectif 4 - Décomposition des différentes contributions	4
1.4	Livrables	4
2	Démarche générale	5
2.1	Principe de moindre action	5
2.2	Equations d'Euler-Lagrange	5
2.3	Décomposition du potentiel	5
2.4	Décomposition des différentes contributions	7
2.5	Reconstruction de la décomposition du potentiel	7
2.5.1	Extraction du potentiel	7
2.5.2	Composantes multipolaires	7
2.6	Trajectoire	11
2.6.1	Principe de fermat	11
2.6.2	Cas $\nu = 2$, quadrupole	11
2.6.3	Equation d'euler lagrange	12
2.6.4	Résolution equa diff(cas sans polariser les apertures)	12
2.6.5	Vérification des résultats	12
2.7	Document Okayama	14
3	Notre production	15
3.1	Structure globale du code	15
3.2	Codes utilisateur	15
3.2.1	Extraction du potentiel	15
3.2.2	Exploitation des données	15
3.3	Programe principal	15
3.3.1	Data	15
3.3.2	Geometry	16
3.3.3	Field calculation	20
3.3.4	Extraction data	20
3.3.5	Fit functions	20
3.3.6	Graphs	20
3.3.7	Multipolar decomposition	20
3.3.8	Paraxial	20



Chapitre 1

Introduction

1.1 Contexte

Dans le cadre du projet multidisciplinaire de 4ème du Génie physique de l'INSA Toulouse, nous devons modéliser les lentilles électrostatiques quadrupolaires d'un FIB (Focused Ion Beam) en utilisant une méthode mathématique appelée la méthode des éléments de frontière. Pour ce faire, nous devons écrire un code Python en utilisant notamment la bibliothèque BEMPP. Afin de pouvoir réaliser cette entreprise, il est nécessaire de comprendre l'optique des particules chargées dans le contexte de lentilles quadrupolaires.

1.2 Présentation du projet

L'optique des particules chargées est une science qui sert à comprendre et à maîtriser la trajectoire de faisceaux électroniques ou ioniques. Cette discipline a permis de créer de nombreux instruments comme notamment les SEM (Scanning electron microscope = microscope électronique à balayage) et les FIB (focused ion beam = faisceau d'ions focalisé) utilisés dans de nombreuses disciplines comme la médecine ou la microélectronique. Plus spécifiquement, un FIB est composé de plusieurs éléments dont notamment des lentilles. En effet, celles-ci permettent de guider la trajectoire du faisceau ionique au sein de l'instrument et de corriger les différentes aberrations de celui-ci. Il est donc primordial de comprendre et maîtriser ces objets afin de rendre le FIB le plus performant possible.

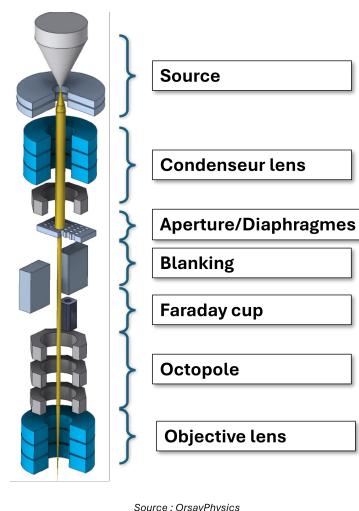
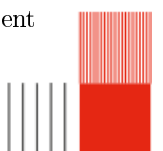


FIGURE 1.1 – Schéma d'un FIB

Cependant, la fabrication de ces instruments peut parfois être longue et coûteuse. Pour pallier ce problème, l'utilisation de modèles s'avère utile. C'est dans ce cadre que notre projet multidisciplinaire s'inscrit et est intitulé : "Modélisation de lentilles multipolaires électrostatiques par Boundary Element



Method”. Comme évoqué précédemment, les lentilles permettent de guider la trajectoire du faisceau ionique. Il existe plusieurs types de lentilles : les lentilles électrostatiques et les lentilles magnétiques. Dans le cas que nous étudions, le FIB est composé de lentilles électrostatiques. Ces lentilles électrostatiques peuvent présenter différentes configurations. On parle en effet de lentilles rondes/monopolaires (un seul potentiel), de lentilles dipolaires (deux potentiels), de lentilles quadrupolaires (quatre potentiels), etc. Notre étude portera plus particulièrement sur les lentilles quadrupolaires.

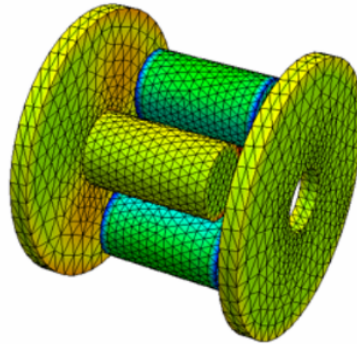


FIGURE 1.2 – Modélisation d’une lentille quadrupolaire électrostatique

Le principe de ce projet multidisciplinaire est de créer un code OpenSource permettant de modéliser des lentilles quadrupolaires électrostatiques. Cette étude s’inscrit dans la continuité de deux projets multidisciplinaires précédents. Le premier projet consistait à prendre en main un code déjà existant permettant de modéliser un cas simple de lentilles électrostatiques (les lentilles de Einzel). Le second avait pour but de créer un algorithme permettant d’optimiser ces lentilles. Notre objectif est donc de continuer ce projet en l’adaptant aux lentilles quadrupolaires. Ce code OpenSource a pour vocation de devenir une ressource utilisable dans le monde de l’optique des particules chargées. Ce projet permettrait notamment à l’entreprise avec laquelle nous travaillons, Tescan, de disposer d’un code facilitant la modélisation de lentilles multipolaires électrostatiques.

1.3 Objectifs du projet

1.3.1 Objectif 1 - Prise en main de BEMPP

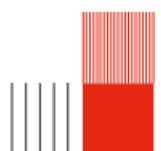
Le premier objectif de ce projet est de se familiariser avec BEMPP, “Boundary element method Python package”, une bibliothèque open-source de calcul par éléments de frontière. Celle-ci permet de résoudre des problèmes électrostatiques, acoustiques et électromagnétiques à l’aide d’une formulation intégrale. Dans notre cas, BEMPP nous permettra de mailler les électrodes des lentilles électrostatiques et de calculer le potentiel en chaque point du volume. Dans un premier temps, nous appliquerons BEMPP dans des cas simples tels que des lentilles rondes. Nous connaissons en effet les résultats attendus car cela a déjà fait l’objet d’un projet multidisciplinaire.

1.3.2 Objectif 2 - Algorithme de la décomposition multipolaire

Le deuxième objectif est de comprendre la méthode de calcul mathématiques qui va nous permettre de décomposer le potentiel donné par BEMPP. Cette étude est essentielle pour retrouver le potentiel électrostatique en n’importe quel point du volume du système. En effet, la décomposition du potentiel est primordiale pour résoudre les équations de la trajectoire, donc connaître le potentiel électrostatique en n’importe quel point du volume, et connaître les aberrations d’un tel système.

1.3.3 Objectif 3 - EXtraction du potentiel

Une fois les outils précédents maîtrisés, nous pourrons les appliquer au cas spécifique des lentilles quadrupolaires électrostatiques. Il sera alors nécessaire de mailler les électrodes du quadrupole et d’appliquer le code Python développé afin d’obtenir le potentiel électrostatique à l’intérieur du système optique. Pour



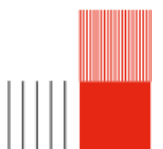
vérifier la justesse de nos calculs, nous pourrions comparer nos résultats à ceux du quadrupole d'Okayama qui a été développé par une équipe de l'université d'Okayama au Japon.

1.3.4 Objectif 4 - Décomposition des différentes contributions

Une fois le potentiel total obtenu, il sera possible d'analyser les trajectoires des particules. Pour cela, nous nous aiderons de l'algorithme de calcul évoqué dans l'objectif 2. Cette approche permettra d'effectuer la décomposition du potentiel Φ , d'en déduire celle de l'indice optique μ , et ainsi d'accéder aux aberrations optiques du système et à la résolution des équations du mouvement. Les équations du mouvement seront résolues à l'aide de méthodes d'intégration numérique de type Runge–Kutta d'ordre 4.

1.4 Livrables

Le premier livrable sera un algorithme de décomposition du potentiel. Le livrable suivant sera un code fonctionnel de modélisation de lentilles quadrupolaires électrostatiques, accompagné d'un manuel d'explication, détaillant le fonctionnement de chaque fonction, des classes et leur utilisation. Dans un dernier temps, nous fournirons une comparaison des résultats de notre code avec ceux du quadrupôle d'Okayama, dont nous connaissons déjà les résultats. De plus, notre code étant open-source, nous fournirons un dépôt GitHub regroupant le code et toute la documentation associée pour qu'il puisse être utilisé.



Chapitre 2

Démarche générale

2.1 Principe de moindre action

Le commencement de ce raisonnement est le principe de moindre action qui s'exprime comme

$$\delta S = \int_{S_0}^{S_1} \vec{p} \frac{\vec{r}}{dS} ds = 0$$

On peut alors montrer que

$$\delta S = \int_{S_0}^{S_1} \mu dz = 0$$

Comme le quadrupole que nous utilisons est composé UNIQUEMENT de lentilles électrostatiques, on peut écrire

$$\mu = \sqrt{\phi(1 + x'^2 + y'^2)}$$

avec μ l'indice optique, ϕ le potentiel électrostatique et x' et y' les dérivées de x et y par rapport à z .

2.2 Equations d'Euler-Lagrange

Les équations d'Euler-Lagrange sont des équations équivalentes au principe de moindre action, sous forme différentielle. On peut ainsi noter

$$\frac{\partial \mu}{\partial x} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial x'} \right) = 0$$

$$\frac{\partial \mu}{\partial y} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial y'} \right) = 0$$

2.3 Décomposition du potentiel

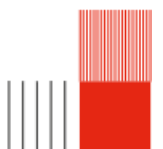
Une fois les équations d'Euler-Lagrange exprimées, il est utile de décomposer le potentiel ϕ afin de séparer les solutions de ces équations en fonction de leur dépendance aux différents ordres de x et y .

On peut ainsi décomposer ce potentiel en série de Fourier et en série de Taylor.

Dans le cadre de notre étude, c'est-à-dire dans le cadre d'un quadrupole, on observe une 2π périodicité du potentiel selon θ . On peut donc décomposer le potentiel comme une série de Fourier. Celle-ci, de part sa périodicité, représente donc les symétries du système. On obtient alors

$$\phi(\mathbf{r}, \theta, z) = \sum_{\nu=0}^{\infty} \phi_{\nu}(\mathbf{r}, \theta, z)$$

avec



$$\phi_\nu(\mathbf{r}, \theta, z) = a_\nu \cos(\nu\theta) + b_\nu \sin(\nu\theta)$$

Ainsi, le potentiel $\phi(\mathbf{r}, \theta, z)$ est composé d'une somme de fonctions dépendantes de $\mathbf{r}$, θ et z . On remarque alors que les variables $(\mathbf{r}, z)$ et θ sont séparées sous la forme suivante :

$$\phi_\nu(\mathbf{r}, \theta, z) = f(\mathbf{r}, z) \times g(\theta)$$

Il est nécessaire que le potentiel électrostatique satisfasse l'équation de Laplace $\Delta\phi(\vec{r}) = 0$. On peut exprimer cette même équation en coordonnées cylindriques comme

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial \phi}{\partial r} \right) + \frac{1}{r^2} \frac{\partial^2}{\partial \theta^2} + \frac{1}{r^2} \frac{\partial^2}{\partial z^2} = 0$$

En injectant $\phi(\mathbf{r}, \theta, z) = \sum_{\nu=0}^{\infty} \phi_\nu(\mathbf{r}, \theta, z)$ dans l'équation précédente, on obtient l'équation suivante

$$\sum_{\nu=0}^{\infty} \left(\cos(\nu\theta) \left(\frac{\partial^2 a_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial a_\nu}{\partial r} + \frac{\partial^2 a_\nu}{\partial z^2} - a_\nu \frac{\nu^2}{r^2} \right) + \sin(\nu\theta) \left(\frac{\partial^2 b_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial b_\nu}{\partial r} + \frac{\partial^2 b_\nu}{\partial z^2} - b_\nu \frac{\nu^2}{r^2} \right) \right) = 0$$

On peut alors montrer que cette équation peut être résolue $\forall \mathbf{r}$, si on vérifie

$$\frac{\partial^2 a_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial a_\nu}{\partial r} + \frac{\partial^2 a_\nu}{\partial z^2} - a_\nu \frac{\nu^2}{r^2} = 0$$

et

$$\frac{\partial^2 b_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial b_\nu}{\partial r} + \frac{\partial^2 b_\nu}{\partial z^2} - b_\nu \frac{\nu^2}{r^2} = 0$$

Il est maintenant utile de développer les fonctions $a_\nu(\mathbf{r}, z)$ et $b_\nu(\mathbf{r}, z)$ en série de Taylor, dans le but de séparer la dépendance des variables $\mathbf{r}$ et z . Dans le cadre de notre étude, on considère des conditions paraxiales, c'est-à-dire des conditions où les rayons sont proches de l'axe optique. On effectue donc le développement en série de Taylor pour $\mathbf{r} = 0$.

Rappel : Une série de Taylor $f(x)$ en $x = b$ s'exprime comme $f(x) = \sum_{n=0}^{\infty} a_n (x - b)^n$.

On peut ainsi écrire

$$A_\nu(\mathbf{r}, z) = \sum_{i=0}^{\infty} a_{i\nu}(z) \times (\mathbf{r})^{\nu+i}$$

et

$$B_\nu(\mathbf{r}, z) = \sum_{i=0}^{\infty} b_{i\nu}(z) \times (\mathbf{r})^{\nu+i}$$

On pose alors $A_{\nu 0}(z) = \Phi_{\nu \mathbf{r}}$ et $B_{\nu 0}(z) = \Phi_{\nu i}$, les composantes de Fourier du potentiel sur l'axe. On peut alors insérer les coefficients de la série de Taylor dans les composantes de la série de Fourier. On obtient alors

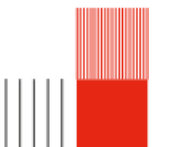
$$A_\nu(\mathbf{r}, z) = \sum_{\lambda=0}^{\infty} \frac{(-1)^\lambda \times (\nu!)}{4^\lambda \times \lambda! \times (\lambda + \nu)!} \times \mathbf{r}^{\nu+2\lambda} \times \Phi_{\nu \mathbf{r}}^{[2\lambda]}(z)$$

et

$$B_\nu(\mathbf{r}, z) = \sum_{\lambda=0}^{\infty} \frac{(-1)^\lambda \times (\nu!)}{4^\lambda \times \lambda! \times (\lambda + \nu)!} \times \mathbf{r}^{\nu+2\lambda} \times \Phi_{\nu i}^{[2\lambda]}(z)$$

On peut alors écrire le potentiel $\phi(\mathbf{r}, \theta, z)$ comme une somme polynomiale avec les dérivées du potentiel pris sur l'axe optique comme coefficients des polynômes. On a alors

$$\phi(\mathbf{r}, \theta, z) = \sum_{\nu=0}^{\infty} \sum_{\lambda=0}^{\infty} \frac{(-1)^\lambda \times (\nu!)}{4^\lambda \times \lambda! \times (\lambda + \nu)!} \times \mathbf{r}^{\nu+2\lambda} \times \left(\Phi_{\nu \mathbf{r}}^{[2\lambda]}(z) \times \cos(\nu\theta) + \Phi_{\nu i}^{[2\lambda]}(z) \times \sin(\nu\theta) \right)$$



2.4 Décomposition des différentes contributions

Une fois le potentiel $\phi(\mathbf{r}, \theta, z)$ décomposé, on peut l'injecter dans l'équation

$$\mu = \sqrt{\phi(1 + x'^2 + y'^2)} = \sqrt{\phi} \sqrt{1 + x'^2 + y'^2}$$

afin de déterminer les différentes contributions qui composent l'indice optique.

Cependant, on remarque que l'indice optique ν dépend à l'ordre 2 des dérivées de x et y . Dans la démarche de décomposer les différentes contributions de l'indice optique, il est nécessaire d'effectuer un développement limité de

$$\sqrt{1 + x'^2 + y'^2}$$

On pose $X = x'^2 + y'^2$ afin d'effectuer un changement de variable.

Rappel : Le développement limité d'une racine carrée est $\sqrt{1 + X} = 1 + \frac{x}{2} - \frac{x^2}{8} + \dots$

On obtient ainsi les différentes contributions de l'indice optique $\mu = \mu_0 + \mu_1 + \mu_2 + \mu_3 + \mu_4 + \mu_5 + \dots$

Les composantes μ_0 , μ_1 et μ_2 composent la partie paraxiale de l'équation tandis que les termes d'ordres supérieurs correspondent aux aberrations. μ_0 correspond à l'axe optique du système. μ_2 correspond au paraxial du système optique.

On note que la paraxialité a une dépendance à l'ordre 2 des variables x et y car en optique, le front d'onde est sphérique. Le front d'onde est une surface d'égale phase d'une onde, c'est-à-dire que les points qui composent cette sphère ont mis le même temps de parcours depuis la source. Ainsi, dans des conditions parfaites, où les aberrations de front d'onde n'existent pas, le front d'onde est parfaitement sphérique, ce qui explique la dépendance d'ordre 2 des variables x et y .

Lorsque le front d'onde n'est plus parfaitement sphérique, autrement dit quand le système connaît une perturbation, tous les points du front d'onde ne vont plus focaliser au même endroit. On obtient ainsi des aberrations. Ces dernières correspondent à tous les termes d'ordres supérieurs à 2 de la somme infinie de ν .

2.5 Reconstruction de la décomposition du potentiel

2.5.1 Extraction du potentiel

A l'aide de la bibliothèque Python BEMPP (*BoundaryElementMethodPythonPackage*), on peut extraire le potentiel $\phi(\mathbf{r}, \theta, z)$ en tout point de l'espace. Comme vu dans la partie précédente, il est important de décomposer le potentiel afin de pouvoir séparer la contribution des différents ordres. Une fois le potentiel extrait, il faut donc trouver les différentes composantes de la décomposition du potentiel.

2.5.2 Composantes multipolaires

Equations

On peut retrouver les composantes multipolaires à l'aide de la formule

$$\Phi_\nu = \frac{2 - \delta_{0\nu}}{\nu!} \left(\frac{\partial^\nu \phi}{\partial \bar{w}^\nu} \right) \Big|_{w=0}.$$

où ν est l'ordre du terme multipolaire et $\delta_{0\nu}$ le symbole de Kronecker.

Pour un développement limité à l'ordre N en $\mathbf{r}$, les valeurs admissibles de λ sont déterminées par la condition

$$2\lambda + \nu \leq N$$

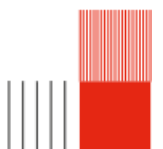
Dans le cadre de ce travail, on se limite à $N = 4$ et $\nu \leq 2$, ce qui conduit aux valeurs suivantes : $\lambda = 0, 1, 2$ pour $\nu = 0$ et $\lambda = 0, 1$ pour $\nu = 1, 2$.

On peut alors calculer les différents ordres des composantes multipolaires :

Cas $\nu = 0$ (monopôle)

On a $\delta_{00} = 1$, d'où

$$\Phi_0(z) = \frac{2 - \delta_{00}}{0!} \frac{\partial^0 \phi}{\partial \bar{w}^0} \Big|_{w=0} = \phi(x, y, z) \Big|_{w=0}.$$



Cas $\nu = 1$ (dipôle)

On a $\delta_{01} = 0$, d'où

$$\Phi_1(z) = \frac{2 - \delta_{01}}{1!} \left(\frac{\partial \phi}{\partial \bar{w}} \right) \Big|_{w=0} = 2 \frac{\partial \phi}{\partial \bar{w}} \Big|_{w=0}.$$

Cas $\nu = 2$ (quadrupôle)

On a $\delta_{02} = 0$, d'où

$$\Phi_2(z) = \frac{2 - \delta_{02}}{2!} \frac{\partial^2 \phi}{\partial \bar{w}^2} \Big|_{w=0} = \frac{1}{2} \frac{\partial^2 \phi}{\partial \bar{w}^2} \Big|_{w=0}.$$

Cas $\nu = 3$ (hexapôle)

On a $\delta_{03} = 0$, d'où

$$\Phi_3(z) = \frac{2 - \delta_{03}}{3!} \frac{\partial^3 \phi}{\partial \bar{w}^3} \Big|_{w=0} = \frac{1}{3} \frac{\partial^3 \phi}{\partial \bar{w}^3} \Big|_{w=0}.$$

Cas $\nu = 4$ (octopôle)

On a $\delta_{04} = 0$, d'où

$$\Phi_4(z) = \frac{2 - \delta_{04}}{4!} \frac{\partial^4 \phi}{\partial \bar{w}^4} \Big|_{w=0} = \frac{1}{12} \frac{\partial^4 \phi}{\partial \bar{w}^4} \Big|_{w=0}.$$

Cas général

On obtient finalement avec $\bar{w} = x - iy$ et $\frac{\partial}{\partial \bar{w}} = \frac{1}{2} \left(\frac{\partial}{\partial x} + i \frac{\partial}{\partial y} \right)$ que

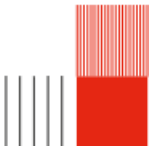
$$\begin{aligned} \Phi_0(z) &= \phi(x, y, z) \Big|_{x=y=0}, \\ \Phi_1(z) &= \left(\frac{\partial \phi}{\partial x} + i \frac{\partial \phi}{\partial y} \right) \Big|_{x=y=0}, \\ \Phi_2(z) &= \frac{1}{8} \left(\frac{\partial^2 \phi}{\partial x^2} - \frac{\partial^2 \phi}{\partial y^2} - 2i \frac{\partial^2 \phi}{\partial x \partial y} \right) \Big|_{x=y=0}, \\ \Phi_3(z) &= \frac{1}{24} \left(\frac{\partial^3 \phi}{\partial x^3} - 3 \frac{\partial^3 \phi}{\partial x \partial y^2} - 3i \frac{\partial^3 \phi}{\partial x^2 \partial y} + i \frac{\partial^3 \phi}{\partial y^3} \right) \Big|_{x=y=0}, \\ \Phi_4(z) &= \frac{1}{192} \left(\frac{\partial^4 \phi}{\partial x^4} - 6 \frac{\partial^4 \phi}{\partial x^2 \partial y^2} + \frac{\partial^4 \phi}{\partial y^4} - 4i \frac{\partial^4 \phi}{\partial x^3 \partial y} + 4i \frac{\partial^4 \phi}{\partial x \partial y^3} \right) \Big|_{x=y=0}. \end{aligned}$$

Ici, $\frac{\partial^\nu \phi}{\partial \bar{w}^\nu}$ représente la dérivée d'ordre ν du potentiel par rapport à la coordonnée transverse w évaluée sur l'axe central $w = 0$.

Comme dans le cas de notre code, on choisit le référentiel (x,y) tel qu'un cosinus décrit le quadrupôle.

On peut donc écrire :

$$\begin{aligned} \Phi_0(z) &= \phi(x, y, z) \Big|_{x=y=0} \\ \Phi_1(z) &= \left(\frac{\partial \phi}{\partial x} \right) \Big|_{x=y=0} \\ \Phi_2(z) &= \frac{1}{8} \left(\frac{\partial^2 \phi}{\partial x^2} - \frac{\partial^2 \phi}{\partial y^2} \right) \Big|_{x=y=0} \\ \Phi_3(z) &= \frac{1}{24} \left(\frac{\partial^3 \phi}{\partial x^3} - 3 \frac{\partial^3 \phi}{\partial x \partial y^2} \right) \Big|_{x=y=0} \\ \Phi_4(z) &= \frac{1}{192} \left(\frac{\partial^4 \phi}{\partial x^4} - 6 \frac{\partial^4 \phi}{\partial x^2 \partial y^2} + \frac{\partial^4 \phi}{\partial y^4} \right) \Big|_{x=y=0} \end{aligned}$$



Applications dans BEMPP

Les dérivées d'ordre supérieur du potentiel par rapport à la coordonnée transverse w , à savoir

$$\frac{\partial \phi}{\partial \bar{w}} \quad \text{et} \quad \frac{\partial^2 \phi}{\partial \bar{w}^2},$$

sont obtenues directement à l'aide des dérivées de BEMPP.

- le gradient du potentiel via `single_layer_gradient`, correspondant à $\nabla \phi$
- la dérivée seconde via `single_layer_2nd_deriv`.

On a pour le terme dipolaire

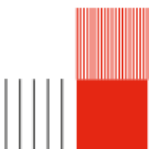
$$\begin{aligned} E_eval[0, :] &= -\frac{\partial \phi}{\partial x} \\ E_eval[1, :] &= -\frac{\partial \phi}{\partial y} \\ E_eval[2, :] &= -\frac{\partial \phi}{\partial x} \end{aligned}$$

On a pour le terme quadripolaire :

$$\begin{aligned} D2_eval[0, :] &= \frac{\partial^2 \phi}{\partial x^2} \\ D2_eval[3, :] &= \frac{\partial^2 \phi}{\partial y^2} \\ D2_eval[1, :] &= \frac{\partial^2 \phi}{\partial x \partial y} \\ D2_eval[5, :] &= \frac{\partial^2 \phi}{\partial z^2} \end{aligned}$$

On a pour le terme hexapolaire :

$$\begin{aligned} D3_eval[0, :] &= \frac{\partial^3 \phi}{\partial x^3} \\ D3_eval[4, :] &= \frac{\partial^3 \phi}{\partial x \partial y^2} \\ D3_eval[1, :] &= \frac{\partial^3 \phi}{\partial x^2 \partial y} \\ D3_eval[6, :] &= \frac{\partial^3 \phi}{\partial y^3} \end{aligned}$$



On a pour le terme octopolaire

$$\begin{aligned}
D4_eval[0, :] &= \frac{\partial^4 \phi}{\partial x^4} \\
D4_eval[4, :] &= \frac{\partial^4 \phi}{\partial x^2 \partial y^2} \\
D4_eval[10, :] &= \frac{\partial^4 \phi}{\partial y^4} \\
D4_eval[1, :] &= \frac{\partial^4 \phi}{\partial x^3 \partial y} \\
D4_eval[7, :] &= \frac{\partial^4 \phi}{\partial x \partial y^3}
\end{aligned}$$

Résultats attendus

Fonctions de fit

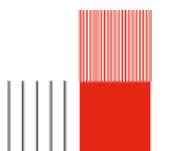
Afin de vérifier résultats que nous obtenons, il est utile de comparer ces derniers aux résultats publiés par S.Okayama dans son papier ; [S. Okayama et H. Kawakatsu, «*Potential distribution and focal properties of electrostatic quadrupole lenses* », *Journal of Physics E : Scientific Instruments*, vol. 11, no 3, p. 211, 1978]. Dans cet article, des fonctions de fit sont utilisées. Il est donc intéressant d'exprimer ces mêmes fonctions dans le cadre de notre étude afin de pouvoir s'assurer de la fiabilité des résultats. Dans le papier d'Okayama, les fonctions de fit sont exprimées comme :

$$\begin{aligned}
\Phi_0(Z) &= V_A k_0(Z) \\
\Phi_2(Z) &= \frac{V_Q}{a^2} k_2(Z) \\
\Phi_4(Z) &= \frac{V_A}{a^4} k_4(Z)
\end{aligned}$$

avec $\Phi_0(Z)$, $\Phi_2(Z)$ et $\Phi_4(Z)$ les composantes du potentiel sur l'axe, $k_0(z)$, $k_2(z)$ et $k_4(z)$ les fonctions de fit, V_A le potentiel des apertures et V_Q la valeur absolue du potentiel des quadrupoles.

De plus, les fonctions de fit sont définies comme

$$\begin{aligned}
k_0(Z) &= a_0 \exp\left(-\frac{Z^2}{b^2}\right) \\
\left\{ \begin{array}{ll} k_2(Z) = 1 & \text{pour } (-Z_0 \leq Z \leq Z_0) \\ k_2(Z) = \exp\left(-\frac{(Z-Z_0)^2}{b^2}\right) & \text{pour } (Z < -Z_0, Z > Z_0) \end{array} \right. \\
\left\{ \begin{array}{ll} k_4(Z) = \frac{a_4}{\left(1 + \frac{Z^2}{b_1^2}\right)^2} & \text{pour } (Z \leq 0) \\ k_4(Z) = a_4 \exp\left(-\frac{Z^2}{b_2^2}\right) & \text{pour } (Z < 0) \end{array} \right.
\end{aligned}$$



2.6 Trajectoire

2.6.1 Principe de fermat

Pour obtenir l'équation de la trajectoire on peut partir du principe de fermat

$$\delta \int \mu ds = 0$$

Dans le cas de l'optique des particules chargés l'indice optique s'écrit comme

$$\mu = \sqrt{\phi(1 + x'^2 + y'^2)}$$

On a donc notre Lagrangian de notre systeme

$$L(x, y, x', y', z) = \sqrt{\phi(1 + x'^2 + y'^2)}$$

2.6.2 Cas $\nu = 2$, quadrupole

Si on prend le cas $\nu = 2$, notre potentiel devient :

$$\phi_2(r, \theta, z) = r^2 \times (\phi_2(z) \times \cos(2\theta)) + \frac{r^4}{12} \times (\phi_2''(z) \times \cos(4\theta))$$

avec $r^2 \cos(2\theta) = x^2 - y^2$

On a donc dans l'approximation paraxiale $\phi_2(r, \theta, z) = \phi_2(z)(x^2 - y^2)$ et donc dans la cas paraxiale :

$$\phi(x, y, z) = \phi_0(z) - \frac{1}{4}\phi_0''(z)(x^2 + y^2) + \phi_2(z)(x^2 - y^2)$$

On peut ensuite injecter ce potentiel dans notre indice optique $\mu = \sqrt{\phi(1 + x'^2 + y'^2)}$, après développement on retrouve les equations d'euler lagrange dans le cas d'un quadrupole)

On peut écrire notre indice optique comme $\mu = \sqrt{\phi(1 + \epsilon)}$ avec $\epsilon : x'^2 + y'^2$. d'après Taylor on a $\sqrt{1 + \epsilon} = 1 + \frac{\epsilon}{2}$

En remplaçant ϵ on peut donc développer l'indice optique comme

$$\mu = \sqrt{\phi} \times \sqrt{1 + x'^2 + y'^2} = \sqrt{\phi} \times (1 + \frac{1}{2}(x'^2 + y'^2)) = \sqrt{\phi} + \frac{\sqrt{\phi}}{2}(x'^2 + y'^2)$$

En remplaçant ϕ par son approximation paraxiale on a :

$$\mu \approx \sqrt{\phi_0 - \frac{1}{4}\phi_0''(x^2 + y^2) + \phi_2(x^2 - y^2)} + \frac{\sqrt{\phi_0 + \dots}}{2}(x'^2 + y'^2)$$

avec

$$\sqrt{\phi_0 - \frac{1}{4}\phi_0''(x^2 + y^2) + \phi_2(x^2 - y^2)} = \sqrt{\phi_0} \sqrt{1 - \frac{\phi_0''}{4\phi_0}(x^2 + y^2) + \frac{\phi_2}{\phi_0}(x^2 - y^2)}$$

En utilisant $\sqrt{1 + \epsilon} \approx 1 + \frac{\epsilon}{2}$, on obtient :

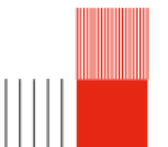
$$\sqrt{\phi_0} \left(1 - \frac{\phi_0''}{8\phi_0}(x^2 + y^2) + \frac{\phi_2}{2\phi_0}(x^2 - y^2) \right) = \sqrt{\phi_0} - \frac{\phi_0''}{8\sqrt{\phi_0}}(x^2 + y^2) + \frac{\phi_2}{2\sqrt{\phi_0}}(x^2 - y^2)$$

$$\mu = \sqrt{\phi_0} - \frac{\phi_0''}{8\sqrt{\phi_0}}(x^2 + y^2) + \frac{\phi_2}{2\sqrt{\phi_0}}(x^2 - y^2) + \frac{\sqrt{\phi_0}}{2}(x'^2 + y'^2)$$

On peut injecter μ dans les equations d'euler lagrange pour retrouver l

$$\frac{\partial \mu}{\partial x} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial x'} \right) = 0$$

$$\frac{\partial \mu}{\partial y} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial y'} \right) = 0$$



Avec :

$$\begin{aligned}\frac{\partial \mu}{\partial x} &= -\frac{\phi_0''}{4\sqrt{\phi_0}}x + \frac{\phi_2}{\sqrt{\phi_0}}x = \left(-\frac{\phi_0''}{4\sqrt{\phi_0}} + \frac{\phi_2}{\sqrt{\phi_0}}\right)x \\ \frac{\partial \mu}{\partial x'} &= \sqrt{\phi_0}x' \\ \frac{d}{dz} \left(\frac{\partial \mu}{\partial x'} \right) &= \sqrt{\phi_0}x'' + \frac{\phi_0'}{2\sqrt{\phi_0}}x'\end{aligned}$$

On obtient finalement l'équation :

$$\left(-\frac{\phi_0''}{4\sqrt{\phi_0}} + \frac{\phi_2}{\sqrt{\phi_0}}\right)x - \left(\sqrt{\phi_0}x'' + \frac{\phi_0'}{2\sqrt{\phi_0}}x'\right) = 0$$

En multipliant par $-\frac{1}{\sqrt{\phi_0}}$, on retrouve la forme :

$$x'' + \frac{\phi_0'}{2\phi_0}x' + \left(\frac{\phi_0''}{4\phi_0} - \frac{\phi_2}{\phi_0}\right)x = 0$$

2.6.3 Equation d'euler lagrange

Il nous reste donc à résoudre les équations différentielles suivantes à l'aide de la méthode Runge Kutta 4 afin d'obtenir les trajectoires

$$\begin{aligned}x''(z) + \frac{\phi_0'(z)}{2\phi_0}x'(z) + \left(\frac{\phi_0''(z)}{4\phi_0(z)} - \frac{\phi_2(z)}{\phi_0(z)}\right)x(z) &= 0 \\ y''(z) + \frac{\phi_0'(z)}{2\phi_0}y'(z) + \left(\frac{\phi_0''(z)}{4\phi_0(z)} + \frac{\phi_2(z)}{\phi_0(z)}\right)y(z) &= 0\end{aligned}$$

On peut trouver les solutions tq :

rayon principal : $x(z_0) = 1, x'(z_0) = 0$

rayon marginal : $x(z_0) = 0, x'(z_0) = 1$

2.6.4 Résolution équation diff(cas sans polariser les apertures)

Dans les deux cas à résoudre une équation différentielle de type

$$x''(z) + \omega_0^2 x = 0$$

On résout cette équation avec la méthode Runge Kutta 4 avec les conditions initiales :

Pour le rayon principal :

$$x(z_0) = 1 \text{ et } x'(z_0) = 0$$

Pour le rayon marginal :

$$x(z_0) = 0 \text{ et } x'(z_0) = 1$$

On peut vérifier le résultat analytiquement

2.6.5 Vérification des résultats

Cas $\omega_0 > 0$

On vérifie les résultats analytiquement

Dans le cas où $\omega_0^2 > 0$, on a (focalisation direction x) :

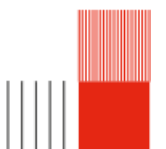
$$x''(z) + \omega_0^2 x(z) = 0.$$

La solution générale est alors :

$$x_m(z) = A \cos(\omega_0(z - z_0)) + B \sin(\omega_0(z - z_0)).$$

On considère le cas du rayon marginal :

$$x(z_0) = 0, \quad x'(z_0) = 1.$$



$$x(z_0) = A = 0.$$

$$x'(z) = -\omega_0 A \sin(\omega_0(z - z_0)) + \omega_0 B \cos(\omega_0(z - z_0)).$$

Ainsi :

$$x'(z_0) = \omega_0 B = 1, \quad \Rightarrow \quad B = \frac{1}{\omega_0}.$$

On obtient finalement la solution analytique du rayon marginal :

$$x_m(z) = \frac{1}{\omega_0} \sin(\omega_0(z - z_0))$$

On considère le cas du rayon principal :

$$x(z_0) = 1, \quad x'(z_0) = 0.$$

$$x(z_0) = A = 1.$$

$$x'(z) = -\omega_0 A \sin(\omega_0(z - z_0)) + \omega_0 B \cos(\omega_0(z - z_0)).$$

$$x'(z_0) = \omega_0 B = 0, \quad \Rightarrow \quad B = 0.$$

On obtient finalement la solution du rayon principal :

$$x_p(z) = \cos(\omega_0(z - z_0))$$

Cas $\omega_0 < 0$

Dans le cas où ω_0 est inférieur à 0 (dans la direction y) on obtient la trajectoire de défocalisation on a

$$y''(z) + \omega_0^2 y(z) = 0.$$

La solution générale est alors :

$$y_m(z) = A \cosh(\omega_0(z - z_0)) + B \sinh(\omega_0(z - z_0)).$$

On considère le cas du rayon marginal :

$$y(z_0) = 0, \quad y'(z_0) = 1.$$

$$y(z_0) = A = 0.$$

$$y'(z) = -\omega_0 A \sinh(\omega_0(z - z_0)) + \omega_0 B \cosh(\omega_0(z - z_0)).$$

Ainsi :

$$y'(z_0) = \omega_0 B = 1, \quad \Rightarrow \quad B = \frac{1}{\omega_0}.$$

On obtient finalement la solution analytique du rayon marginal :

$$y_m(z) = \frac{1}{\omega_0} \sinh(\omega_0(z - z_0))$$

On considère le cas du rayon principal :

$$y(z_0) = 1, \quad y'(z_0) = 0.$$

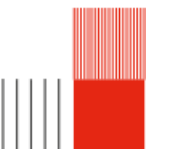
$$y(z_0) = A = 1.$$

$$y'(z) = -\omega_0 A \sinh(\omega_0(z - z_0)) + \omega_0 B \cosh(\omega_0(z - z_0)).$$

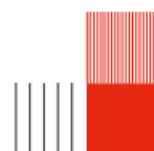
$$y'(z_0) = \omega_0 B = 0, \quad \Rightarrow \quad B = 0.$$

On obtient finalement la solution du rayon principal :

$$y_p(z) = \cosh(\omega_0(z - z_0))$$



2.7 Document Okayama



Chapitre 3

Notre production

3.1 Structure globale du code

3.2 Codes utilisateur

3.2.1 Extraction du potentiel

```
1 from Main_functions import Potential_extraction
2 from Data import Data
3 import os
4 from bempp_cl.core import opencl_kernels
5
6 opencl_kernels.show_available_platforms_and_devices()
7 opencl_kernels.set_default_cpu_device(1,0)
8
9 #Function used to empty the RAM
10 os.environ['PYOPENCL_COMPILER_OUTPUT']='1'
11
12 OUTPUT_DIR = r"C:\Users\zoe\OneDrive - INSA Toulouse\Documents\INSA\4GP\Projet multi\
    projet-multi-code\Quadrupole_model\Files"
13
14 Va = 20000
15 data = Data(4, 5, 21, 19, 2, 15, 2, 13, 3.4934, -0.0299087*Va, -0.18808*Va, +0.18808*Va,
    0, Va, 0.9, 3, 20, OUTPUT_DIR)
16
17 fun = Potential_extraction(data, True)
18
19 fun.mesh()
20 fun.potential_extraction()
21 fun.potential_visualisation()
22 fun.graph_potential_axis()
```

3.2.2 Exploitation des données

3.3 Programme principal

3.3.1 Data

```
1 class Data:
2     """
3     #1 - Distance between the shield ant the aperture
4     #2 - Distance between the aperture and the cylinders (d in Okayama's paper)
5     #3 - Exterior radius of the shield
6     #4 - Inside radius of the shield
7     #5 - Thickness of the shield
8     #6 - Radius of the aperture
9     #7 - Thickness of the aperture
10    #8 - Length of the cylinder (l in Okayama's paper)
11    #9 - Radius of the elements around the axis (a in Okayama's paper)
12    """
13    def __init__(self,
```



```

14         dist_shield_apert: int, dist_apert_quad: int,
15         radius_ext_shield: int, radius_in_shield: int, thickness_shield: int,
16         radius_apert: int, thickness_apert: int,
17         length_cylinder: int,
18         radius_axis: int,
19         pot_electrode: int, pot_apert1: int, pot_apert2: int, pot_shield: int,
20         pot_acceleration: int,
21         MeshSizeMin: int, MeshSizeMax: int, MeshSizeFromCurvature: int,
22         output_dir: str) -> None:
23     self.dist_shield_apert = dist_shield_apert
24     self.dist_apert_quad = dist_apert_quad
25
26     self.radius_ext_shield = radius_ext_shield
27     self.radius_in_shield = radius_in_shield
28     self.thickness_shield = thickness_shield
29
30     self.radius_apert = radius_apert
31     self.thickness_apert = thickness_apert
32
33     self.length_cylinder = length_cylinder
34
35     self.radius_axis = radius_axis
36
37     self.pot_electrode = pot_electrode #quadrupole
38     self.pot_apert1 = pot_apert1 #first aperture (pour le champs rond)
39     self.pot_apert2 = pot_apert2 #second aperture
40     self.pot_shield = pot_shield #shield (0)
41     self.pot_acceleration = pot_acceleration #ion acceleration potential
42
43     self.MeshSizeMin = MeshSizeMin
44     self.MeshSizeMax = MeshSizeMax
45     self.MeshSizeFromCurvature = MeshSizeFromCurvature
46
47     self.output_dir = output_dir
48
49     self.group_id = None
50
51     self.total_length = 2*self.dist_shield_apert + 2*self.thickness_shield + 2*self.
52     dist_apert_quad + 2*self.thickness_apert + self.length_cylinder
53
54     self.coord_cylinder_x_or_y = 2*self.radius_axis
55     self.coord_cylinder_z = self.thickness_shield + self.dist_shield_apert + self.
56     thickness_apert + self.dist_apert_quad
57
58     self.coord_apert_z1 = self.thickness_shield + self.dist_shield_apert
59     self.coord_apert_z2 = self.thickness_shield + self.dist_shield_apert + self.
60     thickness_apert + 2*self.dist_apert_quad + self.length_cylinder
61
62     self.start_shield1 = 0
63     self.end_shield1 = self.start_shield1 + self.thickness_shield
64     self.start_apert1 = self.end_shield1 + self.dist_shield_apert
65     self.end_apert1 = self.start_apert1 + self.thickness_apert
66     self.start_cyl = self.end_apert1 + self.dist_apert_quad
67     self.end_cyl = self.start_cyl + self.length_cylinder
68     self.start_apert2 = self.end_cyl + self.dist_apert_quad
69     self.end_apert2 = self.start_apert2 + self.thickness_apert
70     self.start_shield2 = self.end_apert2 + self.dist_shield_apert
71     self.end_shield2 = self.start_shield2 + self.thickness_shield

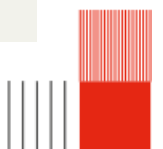
```

3.3.2 Geometry

```

1 import gmsh
2 import os
3 from Data import Data
4
5 #Definition of classes to describe the geometry
6 class Cylinder:
7     """
8     Represents a cylindrical electrode in the GMSH geometry.
9     """
10     def __init__(self, length: int, radius: int, coord_x: int, coord_y: int, coord_z: int
11 ) -> None:
12     """

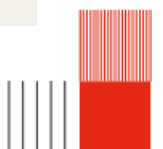
```



```

12         Initialize cylinder parameters.
13
14     Args:
15         length: Length of the cylinder along the Z-axis.
16         radius: Radius of the cylinder.
17         coord_x, coord_y, coord_z: Starting position of the cylinder.
18     """
19     self.length = length
20     self.radius = radius
21     self.coord_x = coord_x
22     self.coord_y = coord_y
23     self.coord_z = coord_z
24
25     self.cyl_tag = None
26     self.cyl_surf = None
27
28
29     def add(self) -> None:
30         """Add the cylinder to the OpenCASCADE model and extract its surface loop."""
31         self.cyl_tag = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0, self.length, self.radius)
32         self.cyl_surf = gmsh.model.occ.get_surface_loops(self.cyl_tag)[1][0]
33
34
35 class Aperture:
36     """
37     Represents an aperture in the GMSH geometry.
38     """
39     def __init__(self, radius_ext: int, radius_in: int, thickness: int, coord_x: int, coord_y: int, coord_z: int) -> None:
40         """
41         Initialize aperture parameters.
42
43         Args:
44             radius_ext: Outer radius of the aperture disc.
45             radius_in: Inner radius (hole) of the aperture.
46             thickness: Thickness of the disc.
47         """
48         self.radius_ext = radius_ext
49         self.radius_in = radius_in
50         self.thickness = thickness
51         self.coord_x = coord_x
52         self.coord_y = coord_y
53         self.coord_z = coord_z
54
55         self.apert_tag = None
56         self.apert_surf = None
57
58
59     def add(self) -> None:
60         """Create the aperture """
61         aperture_out = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0, self.thickness, self.radius_ext)
62         aperture_in = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0, self.thickness, self.radius_in)
63         apert_vol, _ = gmsh.model.occ.cut([(3, aperture_out)], [(3, aperture_in)])
64         self.apert_tag = apert_vol[0][1]
65         self.apert_surf = gmsh.model.occ.get_surface_loops(self.apert_tag)[1][0]
66
67 class Shield:
68     """
69     Represents the shield to fix the potential at 0
70     """
71     def __init__(self, length: int, radius_ext: int, radius_in: int, radius_hole: int, thickness: int, coord_x: int, coord_y: int, coord_z: int) -> None:
72         self.length = length
73         self.radius_ext = radius_ext
74         self.radius_in = radius_in
75         self.radius_hole = radius_hole
76         self.thickness = thickness
77         self.coord_x = coord_x
78         self.coord_y = coord_y
79         self.coord_z = coord_z

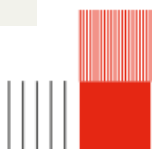
```



```

80
81     self.shield_tag = None
82     self.shield_surf = None
83
84
85     def add(self) -> None:
86         shield_in = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z +
87         self.thickness, 0, 0, self.length - 2*self.thickness, self.radius_in)
88         shield_out = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z,
89         0, 0, self.length, self.radius_ext)
90         shield, _ = gmsh.model.occ.cut([(3, shield_out)], [(3, shield_in)])
91         shield_hole1 = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.length
92         - self.thickness, 0, 0, self.thickness, self.radius_hole)
93         shield_hole2 = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.
94         coord_z, 0, 0, self.thickness, self.radius_hole)
95         shield_vol_1, _ = gmsh.model.occ.cut([(3, shield[0][1])], [(3, shield_hole1)])
96         shield_vol_2, _ = gmsh.model.occ.cut([(3, shield_vol_1[0][1])], [(3, shield_hole2)])
97         self.shield_tag = shield_vol_2[0][1]
98         self.shield_surf = gmsh.model.occ.get_surface_loops(self.shield_tag)[1][0]
99
100 class Mesh_Generation:
101     """
102     Manages the full GMSH workflow: initialization, geometry construction,
103     physical grouping, and mesh generation.
104     """
105     def __init__(self, data: Data, visual: bool) -> None:
106         """
107         data (Data): Object containing all geometric dimensions and mesh settings.
108         visual (bool): If True, launches the GMSH GUI after generation.
109         """
110         self.data = data
111         self.visual = visual
112
113         self.objects = None
114
115     def initialisation(self) -> None:
116         """Initialize GMSH and create a new model."""
117         gmsh.initialize()
118         gmsh.clear()
119         gmsh.model.add("Quadrupole lens")
120
121     def geometry(self) -> None:
122         """Instantiate and add all geometric components (electrodes, apertures, shield)."""
123
124         cylinder1 = Cylinder(self.data.length_cylinder, self.data.radius_axis, self.data.
125         coord_cylinder_x_or_y, 0, self.data.coord_cylinder_z)
126         Cylinder.add(cylinder1)
127
128         cylinder2 = Cylinder(self.data.length_cylinder, self.data.radius_axis, -self.data.
129         coord_cylinder_x_or_y, 0, self.data.coord_cylinder_z)
130         Cylinder.add(cylinder2)
131
132         cylinder3 = Cylinder(self.data.length_cylinder, self.data.radius_axis, 0, self.
133         data.coord_cylinder_x_or_y, self.data.coord_cylinder_z)
134         Cylinder.add(cylinder3)
135
136         cylinder4 = Cylinder(self.data.length_cylinder, self.data.radius_axis, 0, -self.
137         data.coord_cylinder_x_or_y, self.data.coord_cylinder_z)
138         Cylinder.add(cylinder4)
139
140         aperture1 = Aperture(self.data.radius_apert, self.data.radius_axis, self.data.
141         thickness_apert, 0, 0, self.data.coord_apert_z1)
142         Aperture.add(aperture1)
143
144         aperture2 = Aperture(self.data.radius_apert, self.data.radius_axis, self.data.
145         thickness_apert, 0, 0, self.data.coord_apert_z2)
146         Aperture.add(aperture2)

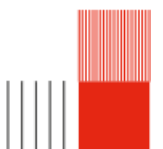
```



```

142         shield = Shield(self.data.total_length, self.data.radius_ext_shield, self.data.
143         radius_in_shield, self.data.radius_axis, self.data.thickness_shield, 0, 0, 0)
144         Shield.add(shield)
145
146         self.objects = aperture1, aperture2, cylinder1, cylinder2, cylinder3, cylinder4,
147         shield
148
149     def creation_mesh(self) -> None:
150         """Synchronize the OpenCASCADE internal CAD representation with the GMSH model."""
151
152         gmsh.model.occ.synchronize()
153
154     def surfaces(self) -> None:
155         """
156         Define physical groups for the surfaces.
157         """
158         aperture1, aperture2, cylinder1, cylinder2, cylinder3, cylinder4, shield = self.
159         objects
160
161         #Outwards orientation of the surfaces' normals
162         gmsh.model.mesh.setOutwardOrientation(aperture1.apert_tag)
163         gmsh.model.mesh.setOutwardOrientation(aperture2.apert_tag)
164         gmsh.model.mesh.setOutwardOrientation(cylinder1.cyl_tag)
165         gmsh.model.mesh.setOutwardOrientation(cylinder2.cyl_tag)
166         gmsh.model.mesh.setOutwardOrientation(cylinder3.cyl_tag)
167         gmsh.model.mesh.setOutwardOrientation(cylinder4.cyl_tag)
168         gmsh.model.mesh.setOutwardOrientation(shield.shield_tag)
169
170         #Adds surfaces on top of the volumes created
171         group_id_apert1 = gmsh.model.addPhysicalGroup(2, aperture1.apert_surf)
172         group_id_apert2 = gmsh.model.addPhysicalGroup(2, aperture2.apert_surf)
173         group_id_cyl1 = gmsh.model.addPhysicalGroup(2, cylinder1.cyl_surf)
174         group_id_cyl2 = gmsh.model.addPhysicalGroup(2, cylinder2.cyl_surf)
175         group_id_cyl3 = gmsh.model.addPhysicalGroup(2, cylinder3.cyl_surf)
176         group_id_cyl4 = gmsh.model.addPhysicalGroup(2, cylinder4.cyl_surf)
177         group_id_shield = gmsh.model.addPhysicalGroup(2, shield.shield_surf)
178
179         self.data.group_id = group_id_apert1, group_id_apert2, group_id_cyl1,
180         group_id_cyl2, group_id_cyl3, group_id_cyl4, group_id_shield
181
182     def mesh(self) -> None:
183         """Configure mesh size and generate the surface mesh."""
184
185         #Size of the mesh
186         #ATTENTION, MeshSizeMin and MeshSizeMax need to be equal
187         gmsh.option.set_number('Mesh.MeshSizeMin', self.data.MeshSizeMin)
188         gmsh.option.set_number('Mesh.MeshSizeMax', self.data.MeshSizeMax)
189         gmsh.option.set_number('Mesh.MeshSizeFromCurvature', self.data.
190         MeshSizeFromCurvature)
191
192         gmsh.model.mesh.generate(2)
193
194     def finalize(self) -> None:
195         #Creates a file .msh
196         mesh_path = os.path.join(self.data.output_dir, "mesh_quadrupole.msh")
197
198         gmsh.write(mesh_path)
199         print("Mesh saved to:", mesh_path)
200
201         if self.visual == True:
202             #Opens a terminal to see the geometry
203             gmsh.fltk.run()
204
205         gmsh.finalize()

```



- 3.3.3 Field calculation
- 3.3.4 Extraction data
- 3.3.5 Fit functions
- 3.3.6 Graphs
- 3.3.7 Multipolar decomposition
- 3.3.8 Paraxial

